AI Agent System Design

From Classical Computer Engineering to Modern Agent Architectures - Part I: Chapters 1-9

Working Draft v0.5 - 2026-06-30

Preface

This document is not an API tutorial and not a transcript of a conversation. It is a technical essay written from the perspective of software engineering and distributed systems. Its goal is to explain why modern AI agents increasingly resemble classical computer systems.

The central thesis is simple: LLMs were the breakthrough, but once LLMs are placed inside real products and workflows, the hard problems start to look familiar again. We need to manage state, reduce expensive remote calls, decide what to load into context, route requests to different compute layers, and design systems that remain observable, reliable, and scalable.

Part I completes the first nine chapters. Chapter 1 establishes the system-level viewpoint. Chapter 2 reframes the LLM as a compute engine. Chapter 3 explains why an agent is better understood as an orchestrator. Chapter 4 separates the responsibilities of memory, tools and planner. Chapter 5 discusses compute/storage separation. Chapter 6 explains why stateless agents resemble microservice design. Chapters 7-9 discuss context engineering, AGENTS.md and retrieval / context routing.

> *Core thesis: the LLM is a new compute engine, while the agent is the orchestrator that manages context, tools, state recovery and resource scheduling around it.*

Table of Contents

Chapter 10	Token Reduction, Distillation and Tiered Compute	Planned
Chapter 11	Agent Production Reliability: Idempotency, State Machines and Replay	Planned
Chapter 12	Multi-Agent, Concurrent Scheduling and Multi-Tenancy	Planned
Chapter 13	Agent Security: Prompt Injection, Sandboxes and Capability Boundaries	Planned
Chapter 14	Toward an Agent Operating System	Planned

Terminology

Term	Meaning in this document	Engineering analogy
LLM	Large language model used for inference	Compute Engine
Agent	System layer that organizes memory, tools, planning and context	Orchestrator / Runtime
Memory	Long-term preferences, project context and task state	Database / Cache
Context	The working set sent to the model for the current request	Working Set / Buffer Pool
Tool	External capability such as files, mail, calendar, GitHub or shell	RPC / API
Planner	Component that decomposes tasks, orders steps and decides whether to continue, retry or escalate	Workflow Engine / Scheduler
Context Builder	Component that selects the current request's working set from memory, files, tool results and task state	Query Optimizer / Buffer Manager
Context Routing	Process of selecting information sources and constructing context based on task, permissions, state and cost	Query Planner / Router
Prompt Index	Index structure that helps an agent locate project knowledge with low context cost	Database Index / Routing Table
Distillation	Moving capability from a large model to a smaller model or fixed	Precomputation / Tiered Compute

	workflow	
Sandbox	An isolated environment that limits an agent's tool, file, network and code execution privileges	OS Process / Container
Idempotency	An execution constraint that makes retries safe from duplicate side effects	Payment Idempotency / Exactly-once Boundary
Replay	Reconstructing agent execution, tool calls and state transitions for debugging, audit and reconciliation	Event Log / Audit Trail

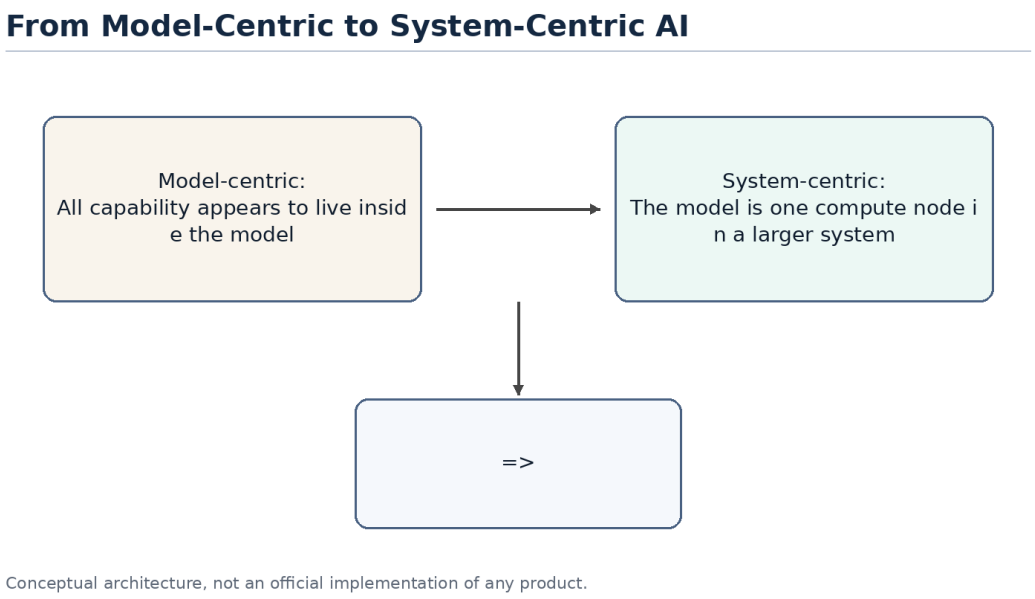

Figure 1

Figure 1. The discussion is shifting from model-centric AI to system-centric agent design.

Chapter 1. Why AI Agents Remind Me of Classical Computer Engineering

Chapter focus: establish the shift from model capability to system architecture.

1.1 The Question

Most public discussions of AI still focus on models: larger models, stronger reasoning, better coding, longer context, and better benchmarks. These improvements matter, but they do not fully explain the recent shift from chatbots to agents. The important change is that AI products are moving from answering questions to completing tasks.

Once the goal becomes task completion, the problem is no longer only model quality. The system must remember long-term state, access external tools, understand the current workspace, decide which historical information matters, and balance cost, latency and reliability. These are not new problems. They are the same type of problems classical computer engineering has been solving for decades.

1.2 Core Thesis

The core thesis of this chapter is that AI agents do not make software engineering obsolete. They make many classical software engineering ideas important again. LLMs introduce a new high-capability compute node, but building reliable, scalable and cost-efficient systems around that node still depends on layering, abstraction, caching, indexing, scheduling, statelessness, and separation of compute and storage.

From this perspective, an agent is not just a smarter chat box. It is closer to an application runtime that organizes user intent, long-term memory, project files, tool calls and model inference into a complete execution process.

1.3 Classical Engineering Concepts Behind the Pattern

Classical computer systems rarely place every responsibility inside a single component. Web systems separate application logic, cache, database, queues, object storage and external APIs. Distributed systems emphasize stateless services, externalized state, retries and horizontal scalability. Database systems emphasize indexes, query optimization, buffer pools and execution plans.

The shared goal behind these techniques is to avoid wasting expensive resources. Databases avoid full table scans. Distributed systems avoid unnecessary cross-service calls. Operating systems avoid unnecessary disk access. Modern agents face a similar problem: do not send every memory, every document and every tool result to the LLM; do not route every task to the most expensive model; do not ask the model to repeat work that can be handled by rules, small models or cached results.

1.4 How This Appears in AI Agents

In the agent world, these engineering ideas appear under new names: Memory, RAG, Context Engineering, Tool Calling, Planner, Router, Distillation, MCP and Multi-Agent systems. They sound new, and some details are new, but many system motivations are familiar.

Memory acts like external state. The context window is the working set of a request. RAG and retrieval resemble loading relevant pages. AGENTS.md can behave like a project-level prompt index. A planner resembles a scheduler. Tool calling resembles RPC. MCP resembles a tool protocol. Distillation and small models resemble moving expensive computation to a lower-cost compute layer.

1.5 Where the Analogy Breaks

Analogies help us think, but they are not proofs. AI agents differ from classical systems in important ways. Traditional APIs are usually deterministic; LLM outputs are probabilistic. A cache hit returns a fixed value; a small model generates an answer and can still be wrong. A database query has a schema; natural language tasks are often ambiguous and open-ended.

Therefore, this document does not claim that an agent is literally a database or that an LLM is literally a CPU. The better claim is that once LLMs become a new compute primitive, many proven computer engineering principles are being reapplied to AI systems.

1.6 Engineering Analysis

From an engineering perspective, the core challenge of AI agents is not only improving a single model response. The challenge is making the whole system complete tasks reliably, continuously and at reasonable cost. This requires three capabilities: context management, resource scheduling and state management.

Context management decides what information enters the model. Resource scheduling decides whether to use a rule, a small model or a large model. State management decides what should be written back to memory, files or external systems. These are system design problems, not just prompt-writing problems.

1.7 Summary

This chapter establishes the document's basic viewpoint: the rapid development of AI agents is not only the result of stronger models. It is the result of combining model capability with classical computer engineering ideas. The LLM is a new high-capability compute node, but the agent system must manage context, state, tools and cost around it.

1.8 Quick Mapping Between Agents and Classical Computer Engineering

Agent concept	Classical engineering concept	Explanation
LLM	Compute Engine	Performs high-capability inference

		but should not own all state
Memory	Database / Cache	Stores long-term preferences, project context and task history
Context Window	Working Set / Buffer Pool	The data loaded for the current request
AGENTS.md	Index / Router	A small entry point that helps locate relevant project knowledge
Tool Calling	RPC / API	Calls external software systems to perform real operations
Planner	Scheduler / Workflow Engine	Breaks tasks into steps and orders execution
Small Model	Tiered Compute	Handles common tasks cheaply and escalates complex tasks

Note: these mappings are thinking tools, not exact equivalences.

Chapter 2. LLM as a New Compute Engine

Chapter focus: position the LLM as a compute node, not as the entire system.

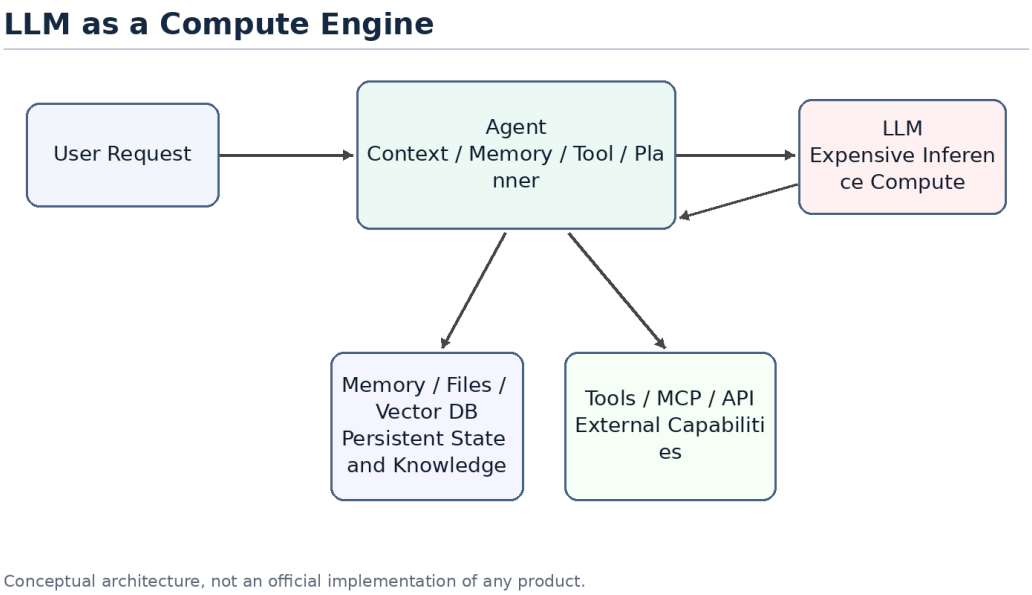

Figure 2

Figure 2. LLM as an expensive compute engine inside an agent system.

2.1 The Question

People often mix products such as ChatGPT, Claude or Gemini with the underlying LLM. A product is not just a model. It usually includes a user interface, an agent layer, memory, tool calling, permissions, file handling, monitoring, billing and safety policies. The LLM is only the most important and often the most expensive compute node inside the larger system.

Thinking of the LLM as a compute engine helps clarify agent architecture. The model does not inherently store your long-term memory or your project history. It performs inference based on the context provided in the current request. The agent layer is responsible for recovering state, selecting information and calling tools.

2.2 System Role of the LLM

In classical systems, expensive compute services are rarely exposed directly to all business logic without a management layer. Search engines have indexing services. Databases have optimizers. GPU inference clusters may have batching, caching and routing layers in front of them.

LLMs should be understood in a similar way. They can process language, code, reasoning and planning, but that does not mean all state should live inside the model or every task should go

directly to the largest model. An LLM is a powerful but expensive remote compute service. Every call consumes input tokens, output tokens, latency and GPU resources.

2.3 Stateless Compute

From the perspective of a single inference request, an LLM can mostly be treated as stateless compute. It does not permanently remember a specific user in its weights just because it had a conversation with that user in the previous turn. If the next request does not include the relevant context, the model cannot reliably know what happened before.

This means that when a product appears to remember you, the memory usually comes from the agent layer. The agent retrieves memory, recent conversation, file snippets and tool results, then composes them into the prompt. The model appears to remember because the required state has been restored for that request.

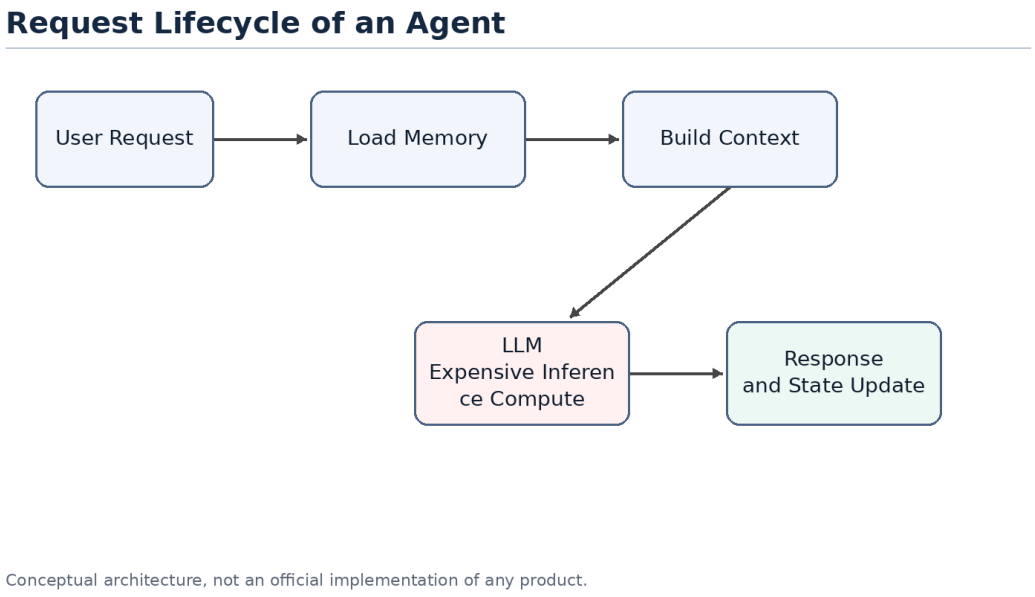

Figure 3

Figure 3. A stateless LLM requires the agent to restore context for each request.

2.4 LLM API as an Expensive Remote Call

Calling an LLM API is similar to calling an expensive remote service. It is slower than a local function call, more expensive than most database reads, and less deterministic than traditional APIs. Therefore, a central goal of agent design is reducing unnecessary calls to large models.

This mirrors classical system optimization. We used to reduce cross-service RPC calls, database queries and disk IO. Now we also reduce unnecessary LLM calls, redundant context and repeated reasoning. Tokens, context length, model tier and call count are becoming new performance metrics for AI systems.

2.5 Token Cost and Compute Cost Are Not the Same

It is important to distinguish token count from compute cost. This distinction is easy to get wrong. For example, people sometimes say that they want to use distillation to reduce token usage. Strictly speaking, distillation usually does not reduce the number of tokens. It reduces the compute cost of processing the same tokens. A small model and a large model may receive the same number of tokens, but the small model has a lower per-token cost.

The cost of an agent model call can be decomposed with a simple formula:

```
total cost ~= call count x tokens per call x cost per token
```

These three factors map to different optimization families. Planner limits, cache hits and task merging reduce call count. Context engineering, retrieval, pruning, summarization, prompt caching and compressed tool outputs reduce tokens per call. Distillation, small models, routing and cascades reduce cost per token.

Dimension	**Reducing Token Count**	**Reducing Per-Token Compute Cost**
Optimization target	Number of input and output tokens per request	Compute and price required to process the same tokens
Distributed-systems analogy	Reduce RPCs, shrink payloads and reduce round trips	Move work to a cheaper compute tier, similar to hot/cold tiering or service degradation
Typical techniques	Context engineering, RAG, pruning, summarization, prompt caching, compressed tool outputs and planner iteration limits	Distillation, small models, routing and cascades, where a cheap model tries first and uncertain cases escalate
Need for training/data	Usually no; engineering and configuration can be enough	Distillation needs labeling, training, evaluation and deployment; routing or existing small models may not
Implementation cost and speed	Low cost and fast feedback; prompt, retrieval and cache changes can help immediately	Distillation is high cost; routing is medium cost; benefits depend on stable tasks and evaluation
Main risk	Over-pruning loses context and makes the model answer from incomplete information	Wrong-tier routing: simple tasks hit large models and waste money, or hard tasks are sent to weak models and fail
Source of uncertainty	Retrieval recall, summarization fidelity and whether tool-output compression damages information	Small-model generalization boundaries and confidence estimation quality
Priority for independent developers	Do this first; low barrier and usually no training pipeline	Consider later, after traffic and data become stable; distillation is not the starting move

Therefore, a more precise statement is that distillation primarily reduces the use of expensive large models, not necessarily the number of tokens. Token usage decreases only when distillation shortens the workflow, reduces repeated planning calls, or replaces several large-model calls with a smaller computation. In that case, the saved tokens come from removed calls and intermediate context, not from distillation directly compressing the tokens inside one request.

This distinction matters in real products. For an independent developer, distillation is a heavy tool: it requires labeled data, a training pipeline, evaluation sets, deployment and model hosting. The token-reduction family looks more like ordinary systems optimization: cache stable prefixes, prune irrelevant context, compress tool outputs, limit planner iterations and avoid unnecessary large-model calls through routing. These can usually work without training.

So if the goal is to reduce cost first, the practical order is usually to exhaust call-count and token-count reductions before moving to routing, small models and distillation. Distillation belongs later, especially for workflows with stable traffic, stable task distributions and enough examples.

2.6 Analogy to Classical Compute

As a compute engine, an LLM can be compared to a database execution engine, a remote compute service or a GPU inference cluster. It is powerful, but it should not carry every responsibility in the system. A database does not own business workflow. A CPU does not perform operating-system scheduling by itself. Similarly, an LLM should not be viewed as the entire agent system.

This viewpoint explains why the agent layer matters. The agent does not replace the model; it ensures that the model is called at the right time, with the right context, and at a reasonable cost.

2.7 Summary

This chapter positions the LLM as a new high-capability compute engine. It is powerful, expensive, mostly stateless and usually accessed through an API. This explains why memory, context, tools and planning should be viewed as parts of the agent system rather than as properties of the model alone.

Chapter 3. Agent as an Orchestrator

Chapter focus: the agent is the orchestration layer that routes work to the right resource.

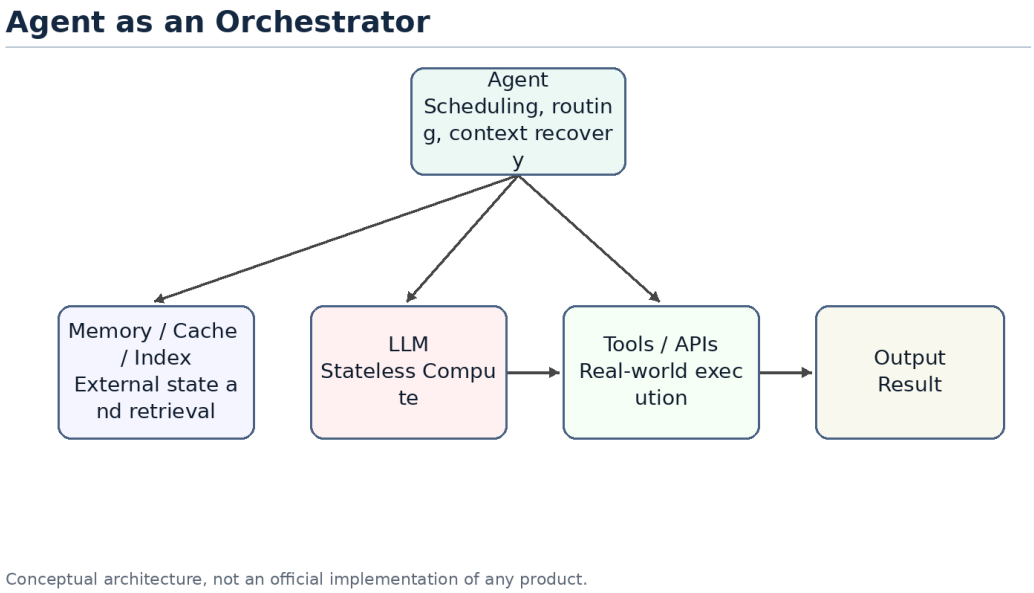

Figure 4

Figure 4. The agent as an orchestrator rather than a chatbot.

3.1 The Question

If the LLM is the compute engine, what is the agent? A common misunderstanding is to treat an agent as a wrapper around a large model or as a chatbot with tools. That description captures part of the surface behavior, but it misses the architectural role.

A better definition is that an agent is an orchestrator. It receives a user goal, restores relevant state, selects context, decides whether to call tools, chooses which model or compute layer to use, and organizes multiple steps into an executable workflow.

3.2 Core Components of an Agent

A typical agent contains several logical components. Memory stores long-term preferences, project background and task history. A context builder decides what should enter the current prompt. A planner decomposes the user goal into steps. A tool layer calls external systems. A router or scheduler decides whether to use rules, small models or large models. An evaluator or guardrail decides whether the result is reliable enough.

These components may be physically distributed across multiple services or hidden inside a product. But logically, they are part of the agent layer, not part of the LLM itself.

3.3 Typical Execution Flow

When an agent receives a request, it should not immediately send the raw user message to the largest model. A more efficient flow is to classify the task, retrieve relevant memory or files, build context, then select a compute resource. If a simple rule can solve the task, no model is needed. If the task matches a common pattern, a small model may be enough. If the task is complex or risky, the agent escalates to a large model.

This is the difference between a chatbot and an agent. A chatbot tends to map one input to one response. An agent receives a goal and organizes a set of computations and operations to complete it.

3.4 Agent, API Gateway and Scheduler

From a systems perspective, an agent resembles a mixture of API gateway, workflow engine and scheduler. It exposes a unified interface to the user while connecting internally to models, tools, memory, files and external services. It also chooses an execution path based on task complexity and cost.

That is the role of an orchestrator. It may not perform all computation by itself, but it decides which resources participate, in what order they participate, how results are combined, and whether failures should trigger retries, fallbacks or escalation.

3.5 Tiered Compute: Rule, Cache, Small Model, Large Model

Agent scheduling can be abstracted as tiered compute. The lowest layer is rules: lowest cost and fastest, but limited coverage. The next layer is cache or retrieval, which reuses existing results or finds relevant context. Above that is a small model. A small model does not store fixed answers like a cache; it stores capabilities learned through training or distillation. The highest layer is the large model, which is the most capable and the most expensive.

This resembles multi-level caches, hot/cold storage tiers and service degradation strategies in traditional systems. A good agent should not route every request directly to the most expensive model. It should solve as many requests as possible at cheaper layers.

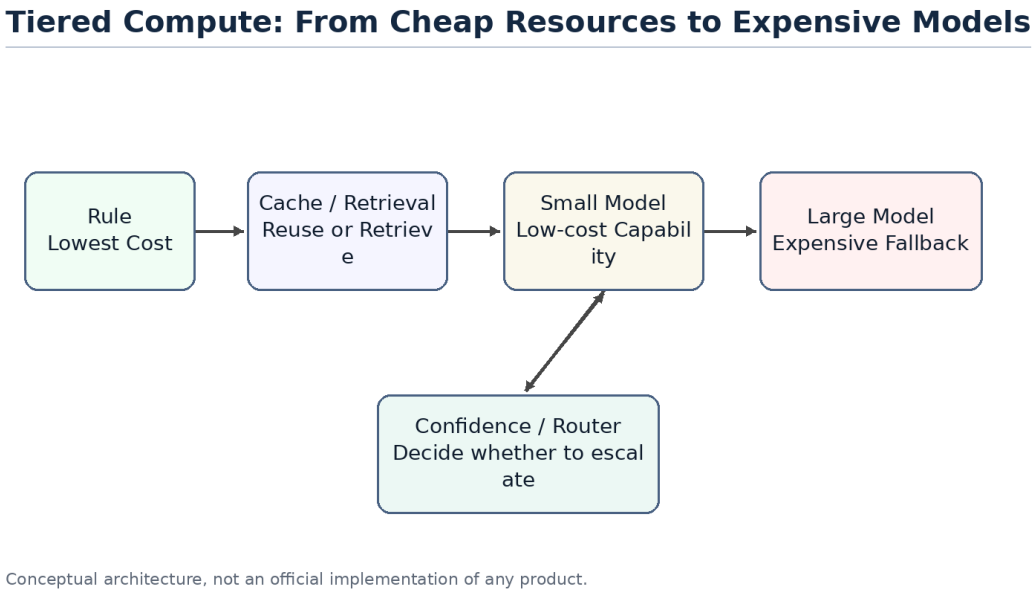

Figure 5

Figure 5. Tiered compute prevents every request from hitting the largest model.

3.6 Why a Small Model Is Not Just a Cache

Small models are sometimes compared to caches, but that analogy needs refinement. A normal cache stores results. On a hit, it returns a fixed value. A small model stores capability. It still performs inference based on the input. It cannot guarantee a fixed answer like Redis, but it can generalize to problems that were not explicitly seen during training.

A better phrase is capability cache. Distillation compresses behavior observed from a larger model into a lower-cost compute layer. It does not eliminate computation; it moves computation from an expensive model to a cheaper model.

3.7 Risks and Limits of Agent Orchestration

An agent as orchestrator introduces new risks. A bad router may send simple tasks to an expensive model or complex tasks to an underpowered model. Bad memory retrieval can make the model reason over the wrong context. Multi-step tool calls can amplify small mistakes. Probabilistic model behavior makes traditional testing incomplete.

Therefore, agent systems need observability, logs, audits, replay, evaluation sets and escalation policies. This again shows that agents are not just conversation interfaces. They are software systems that need system engineering discipline.

3.8 Summary

This chapter positions the agent as an orchestrator. It is not the model itself. It is the system layer that organizes memory, tools, planning, context and resource scheduling around the

model. This prepares the ground for later chapters: why memory resembles storage, why AGENTS.md resembles an index, why distillation resembles tiered compute, and why multi-agent systems increasingly resemble distributed systems.

Chapter 4. Memory, Tools and Planner

Chapter focus: separate the responsibilities that are often mixed inside an agent.

4.1 The Question

The previous chapters positioned the agent as an orchestrator. But it is not enough to say that an agent organizes memory, tools, planning and context. In real system design, the hard part is not whether these components exist. The hard part is whether their responsibilities are separated.

Many agent products put everything into one large prompt: user history, project background, tool results, planning steps, errors and the final answer. This can make a demo work, but it creates long-term problems. State becomes hard to control, tool calls become hard to audit, and planning becomes hard to replay.

This chapter asks a more precise question: what should memory store, what should tools do, and what should the planner decide? These components roughly map to storage, external APIs and workflow scheduling, but they are not the same thing.

4.2 Memory Stores State; It Does Not Think for the Model

Memory stores reusable state. It can include user preferences, project background, historical decisions, task progress, file summaries and long-lived facts. It should not be understood as a magic layer that makes the model smarter. It is external state managed by the agent system.

From an engineering perspective, memory is closer to a database or cache than to the model itself. The model is still stateless compute for each inference call. The agent reads relevant information from memory, then puts only part of it into the current context. In other words, memory can store a lot, but the model only sees the working set selected by the context builder.

This distinction matters. If memory is designed as an endlessly growing chat transcript, the system soon becomes a mechanism for pushing more history into the prompt. A better design separates stable long-term facts, recent task state and temporary tool results. Only results that are worth reusing should be summarized and written back into long-term memory.

4.3 Tools Produce Effects; They Do Not Own Long-Term Meaning

Tools connect the agent to the outside world. They can read files, send email, inspect calendars, access GitHub, run shell commands, call databases or operate business systems. The defining property of a tool is that it may produce real side effects.

This makes tools fundamentally different from memory. Memory is state managed by the agent. A tool is an external capability. A tool call may create an issue, send a message, modify a file, start a payment, update an order or delete a resource. These actions cannot be treated as just another piece of model output. They are system actions that need auditability, retry rules and sometimes rollback or reconciliation.

A tool layer therefore needs to handle at least four concerns: permissions, parameter validation, execution results and error semantics. The agent should record which tool was called, which arguments were passed, what result came back, whether side effects occurred, and whether a failure can be retried.

4.4 The Planner Decides Steps; It Should Not Execute Everything

The planner decomposes a user goal into executable steps and decides their order. It answers the question "what should happen next?" It should not own tool internals or persistence logic.

This resembles a workflow engine or scheduler. A workflow engine does not itself send email, write databases or run queries. It decides when those operations happen, what prerequisites they require, whether failures should be retried and whether human confirmation is needed. An agent planner should play a similar role. It organizes work; it should not hide all business logic inside a prompt.

A common mistake is giving the planner too much freedom. A model can generate a long plan, but long plans create more failure points, higher cost and weaker auditability. A safer design asks the planner for short plans and re-evaluates after each step based on tool results.

4.5 The Context Builder Connects the Components

Memory, tools and planner need another component that is easy to overlook: the context builder. It turns long-term state, current task state, tool results and planning steps into the context for the next model call.

The context builder is not just string concatenation. It decides which memories are relevant, which tool results should remain visible, which steps are already done, which errors the model must know about, and which information should be compressed or dropped. It resembles a mixture of query optimization and working-set management.

Without a context builder, memory becomes uncontrolled text, tool results become a growing log, and the planner becomes a long list without feedback. Much of agent reliability depends on whether this layer clearly manages what the model sees in the current request.

4.6 Responsibility Boundaries

Component	Main responsibility	Should not do	Engineering analogy
Memory	Store long-term state, preferences, project context and task history	Reason for the model or send all history into the prompt	Database / Cache
Tool	Call external systems and return results, with side effects when necessary	Own long-term semantics or hide side effects	RPC / API
Planner	Decompose goals, order steps and decide whether to continue or escalate	Execute external operations directly or produce an unchangeable	Workflow Engine / Scheduler

		long plan	
Context Builder	Select the working set for the current request	Concatenate unlimited information	Query Optimizer / Buffer Manager

The point of this table is not naming. It is preventing responsibility leakage. Memory should not become a prompt junk drawer. Tools should not be treated as pure functions when they have side effects. The planner should not become an unauditable natural-language script. The context builder should not be just a string builder.

4.7 Failure Modes

Blurry responsibility boundaries produce predictable failure modes.

First, memory pollution. Incorrect facts, stale information or temporary tool results are written into long-term memory and then reused in future tasks.

Second, uncontrolled tool side effects. The model calls the same tool repeatedly, creating duplicate issues, sending duplicate messages or modifying files twice, while the system lacks idempotency keys and call logs.

Third, runaway planning. The model creates a long plan without checkpoints or feedback from tool results. When the task fails, it is unclear which step caused the failure.

Fourth, context bloat. All history and tool results are sent to the model. Token cost rises, and important facts are buried under noise.

4.8 Summary

This chapter separated memory, tools, planner and context builder. Memory is the state layer. Tools are the external capability layer. The planner is the orchestration layer. The context builder manages the working set for the current request.

This separation prepares the next chapters. Chapter 5 explains why compute and storage should be separated. Chapter 6 explains why the agent itself should remain as stateless as possible. Later chapters on production reliability will return to tool side effects, idempotency, state machines and replay.

Chapter 5. Compute / Storage Separation

Chapter focus: explain why an agent architecture should not merge model, state and knowledge into one blob.

5.1 The Question

In classical system design, separation of compute and storage is a basic principle. The compute layer executes logic. The storage layer persists state. Web services do not keep all user state in process memory. Databases do not own every business workflow. Distributed systems do not assume that a compute node permanently holds the complete context.

AI agents face the same problem. The LLM is a high-capability compute engine, but it is not long-term storage. An agent may call a model, read memory, retrieve documents, invoke tools and update state. If all of these responsibilities are pushed into a prompt or into model weights, the system becomes expensive, fragile and hard to maintain.

The core question of this chapter is: what should count as compute, and what should count as storage in an agent system? Why does separating them make agents easier to scale, debug and cost-control?

5.2 The LLM Is Compute, Not Storage

The LLM's role is to reason, generate and judge based on the current input. It can handle complex language, code and planning tasks, but it should not be treated as the place where all history and business state live.

From the perspective of one call, the LLM is a remote compute service. It reads the prompt, performs inference and returns a result. On the next call, if the relevant state is not provided in the prompt, it cannot reliably recover what happened before. Even a long context window does not make the model a database. A long context window is a larger working set, not durable storage.

Therefore, pushing everything into context is not compute/storage separation. It is moving storage temporarily into the compute request. That can work for small ad hoc tasks, but it is not a good long-term system design.

5.3 Memory, Files and Indexes Are Storage

The storage layer inside an agent system can take many forms: structured databases, vector indexes, file systems, object storage, caches, event logs, user-preference tables and task-state tables. Their shared responsibility is to persist state and let the agent read it when needed.

Memory is only one part of storage. Project files, tool results, user configuration, permission policies, task progress and audit logs all belong to the broader storage layer. Calling all of them memory blurs the design. A more precise statement is: memory is the state view used to restore model context, while storage is the full set of durable system state.

This distinction affects architecture. User preferences may go into memory. Order state should live in a business database. Tool-call logs should go into an audit log. Large documents should live in a file system or object store. Retrieval chunks should live in an index. These should not all become prompt text.

5.4 Context Is the Working Set of a Compute Request

Compute/storage separation does not mean compute needs no state. Every compute request needs a selected subset of state. That subset is context.

Context is like the pages loaded into a database buffer pool for a query, or the working set of an operating-system process. It comes from storage, but it is not storage itself. Before each model call, the agent selects a small amount of information from memory, files, indexes and tool results, then organizes it into input the model can use.

This explains why context engineering is central to agent systems. It is not merely prompt style. It is the data-loading strategy between storage and compute. Load too little and the model lacks context. Load too much and cost rises, noise increases and important facts get buried.

5.5 RAG Is a Read Path, Not the Whole System

RAG is often treated as the center of agent architecture, but more precisely it is a read path from storage into context. Retrieval finds relevant document chunks and places them into the model context so the LLM can answer using external knowledge.

This is useful, but it is not complete state management. RAG mostly solves the problem of finding relevant content in a large text corpus. It does not automatically handle task state, permissions, side effects, retries, audits, long-term preferences or business consistency. A production agent cannot manage all state through RAG alone.

RAG should therefore be placed in the right architectural role: one read path from storage. It sits alongside database queries, file reads, cache hits and tool-result loading. The agent must decide when to use retrieval, when to query structured state, when to read files and when to call tools.

5.6 The Write Path Matters Too

Many agent discussions focus on how to put information into the prompt, but ignore what should be written back to storage. For a long-running system, the write path is just as important.

After completing a task, the agent may need to write back several kinds of results: user preferences, task progress, decision records, tool-call summaries, errors, evaluation results and audit logs. Not every model output should be persisted. Before writing back, the system must decide whether the information is stable, true, reusable and allowed to be stored.

This resembles write paths in classical systems. Database writes need schemas, constraints and transactions. Cache writes need expiration policies. Event logs need ordering and immutability. Agent memory writes need similar discipline. Otherwise the system will permanently store hallucinations, temporary guesses and stale state.

5.7 Engineering Benefits of Separation

Design problem	Merged design	Separated design
Long-term state	Put everything into the prompt or chat history	Store it in databases, files, memory or event logs
Current context	Concatenate history without limits	Select a working set from storage
Cost control	Context grows indefinitely	Control tokens through indexes, caches and pruning
Debuggability	Unclear which state influenced the answer	Inspect which memory, files and tool results were loaded
Consistency	Treat model output as fact	Validate, structure and permission-check before writing back
Scalability	Carry large history in every request	Scale storage independently and load on demand

The main benefit of compute/storage separation is control. The model remains powerful, but it is no longer the container for all system state. Where state is stored, when it is read, how much is loaded and when results are written back become designable and auditable engineering decisions.

5.8 Summary

This chapter separated compute and storage in agent systems. The LLM is compute. Memory, files, indexes, databases and logs form storage. Context is the working set of a compute request. RAG is one read path, not the whole system.

This viewpoint explains why long context does not solve everything. Long context expands the working set of one request, but it does not replace durable state, indexes, write-back policies or consistency control. Chapter 6 extends this idea: once state is externalized, the agent itself can behave more like a stateless service.

Chapter 6. Stateless Agents

Chapter focus: explain why the agent layer should be mostly stateless, and how externalized state improves reliability.

6.1 The Question

Chapter 5 discussed compute/storage separation. This chapter takes the next step: if the LLM is stateless compute and memory/files are external storage, what kind of state should the agent layer itself hold?

One intuitive implementation is to let the agent process keep a large amount of session state: the current plan, historical tool results, user preferences, execution progress and error context. This is simple at first, but it creates the same problems as stateful services in classical microservice systems: scaling, restarts, retries and recovery become harder.

A more robust design is to make the agent as stateless as practical. It may hold temporary execution context during a request, but long-lived state should live in memory, databases, file systems, task queues or event logs. Then agent instances can scale horizontally, retry safely, be replaced and support replay.

6.2 Stateless Does Not Mean No State

A stateless agent does not mean the system has no state. It means the state should not depend on the memory of one agent process. State still exists, but it is stored externally and restored through explicit read paths.

This is similar to web services. A stateless web service still handles login, carts and orders. It simply does not keep those states only in the memory of one server. When a request arrives, it restores state from cookies, a session store, a database or a cache. When the request completes, it writes necessary results back to external systems.

Agents should work similarly. Before a task runs, the agent can read user memory, project files, task state and tool logs. During execution, it builds context. After execution, it writes stable results back. As long as that state is not tied to one process, the agent can behave more like a stateless service.

6.3 Why Agents Drift Toward Stateful Mud

Agents easily become stateful mud because natural-language tasks have blurry boundaries. What the model said in the previous turn, what tools returned, what the user changed temporarily and where the planner currently is all feel like things the system should remember.

Without explicit design, the simplest implementation is to put all of that into a runtime object and keep appending. This is convenient in the short term, but it creates long-term problems. A process restart loses task state. Multiple instances disagree. Retries cannot tell whether a tool already executed. A page refresh cannot fully recover context. Logs and real state diverge.

This is why agent systems cannot focus only on prompts. A prompt is the input for one model call. It is not the only source of system state. Real state needs structure, lifecycle and write-back rules.

6.4 Basic Forms of External State

A stateless agent puts different kinds of state into different external systems.

State type	Good location	Explanation
User preferences	Memory / user configuration table	Long-lived, but must be editable and deletable
Project background	File system / document index / memory summary	Searchable, but not all loaded into context
Task progress	Task state table / workflow state	Used for recovery, progress display and retries
Tool calls	Audit log / event log	Records arguments, results, side effects and errors
Temporary context	Request memory / short-lived cache	Used only during the current task lifecycle
Final artifacts	Files, databases or external business systems	Persisted according to task type

The key idea is lifecycle. Not all state should be stored forever, and not all state should go into memory. Temporary context can disappear when the task ends. Tool-call logs may need long retention. User preferences need edit and delete paths. Task state must be structured enough to support recovery and replay.

6.5 Scalability Benefits

Once the agent is stateless, the system can scale like a normal microservice. Multiple agent instances can handle requests because they do not depend on local long-term memory. If one instance fails, another can recover the task from external state. When traffic grows, the system can add instances. When the model or agent code changes, rolling deployment becomes easier.

This matters especially for agents because agent tasks may be slow. A task can involve multiple model calls, multiple tool calls and waiting for external systems. The system cannot assume that one process will stay alive forever, or that the whole task will complete inside one short connection. Task queues, state tables and event logs are more reliable than in-process objects.

Statelessness also makes multi-tenancy easier. State for different users and projects can be managed through tenant IDs, permission policies and storage isolation, rather than relying on an agent process to remember whom it is serving.

6.6 Retry and Recovery Benefits

The most dangerous part of agent execution is retry. Model calls can be retried. Retrieval calls can be retried. But tool calls may already have produced side effects. If agent state only lives in memory, the system may not know whether a failure happened before or after the side effect.

External state makes retries controllable. Each task step can have a state such as pending, running, succeeded, failed or needs_review. Each tool call can have an idempotency key, argument digest, result summary and side-effect record. Before retrying, the agent checks state and avoids repeating operations that already succeeded.

This is the same discipline used in production systems. Payment systems, order systems and message queues store whether an action has already happened. They do not rely on process memory. Once agents start calling real tools, they need the same discipline.

6.7 The Cost of Statelessness

Statelessness is not free. Externalizing state requires more infrastructure: databases, caches, object storage, task queues, event logs and permission systems. Each request also needs state restoration before context construction, which increases latency and complexity.

External state also exposes schema design questions. Which fields should be structured? Which content should remain raw text? Should tool results be saved fully or summarized? Should memory writes require human confirmation? These questions cannot be avoided by putting everything into the prompt.

The goal is therefore not to over-engineer every agent from the beginning. It is to choose the right tradeoff between reliability and implementation cost. An early product can start with a lightweight state table and simple logs. As tasks become more complex, it can add stricter workflow state, audit logs and replay mechanisms.

6.8 Summary

This chapter treated the agent layer as a mostly stateless service. Stateless does not mean no state. It means state is not tied to one agent process. Memory, task state, tool logs and final artifacts should be managed through external storage.

This design makes agents easier to scale, restart, retry, recover and audit. It also leads into later chapters. Context engineering will explain how to construct the current working set from external state. Production reliability will return to idempotency, state machines, replay and reconciliation.

Chapter 7. Context Engineering and Query Optimization

Chapter focus: treat context engineering as data loading and query optimization, not just prompt writing.

7.1 The Question

The previous chapters positioned the LLM as compute, memory/files/indexes as storage, and context as the working set of one compute request. The next question is: before each model call, which information should be loaded?

Many people treat context engineering as writing a better prompt. That is part of it, but from a systems perspective it is closer to query optimization. A database does not send an entire table into the execution engine. An operating system does not load the entire disk into memory. An agent should not send every chat message, every document and every tool result to the model.

The core goal of context engineering is not more context. It is more relevant, cheaper and more verifiable context. It balances recall, precision, cost, latency and reliability.

7.2 Context Is the Execution Input for a Model Call

A model call looks like a function call, but the input is not simply the user's raw message. The input is context constructed by the agent. It may include system instructions, user goals, memory summaries, file snippets, tool results, planning state, errors and output format requirements.

Context therefore resembles the execution input after query planning. A database execution engine does not see the entire business world. It sees data and operations selected by the optimizer. Similarly, the LLM does not see the entire project. It sees the working set selected by the context builder.

This means context quality directly affects model quality. A strong model will still guess if key facts are missing. It will be misled if wrong facts are included. If context is too long, important information is diluted by noise and cost increases.

7.3 The Query Optimization Analogy

A database optimizer estimates available indexes, join choices, filter selectivity and scan cost. A context builder needs similar judgment: which memories are relevant, which file snippets should be loaded, whether tool results are still valid and which old context can be dropped.

The analogy is not perfect. Databases have schemas, statistics and deterministic execution plans. Natural-language tasks are blurrier and relevance is harder to estimate. But the engineering motivation is the same: do not waste expensive compute on irrelevant data.

If the LLM is an expensive execution engine, context engineering is data selection, filtering, ordering and compression before the call. It decides what the model sees and what it does not see.

7.4 Inputs to the Context Builder

Input source	Typical content	Main risk	Optimization problem
User request	Current goal, constraints, preferences	Ambiguity and goal drift	Clarify intent and extract the task
Memory	User preferences, project context, decisions	Stale or polluted memory, over-recall	Select stable and relevant state
File system	README, code, docs, config	Too many files, version mismatch	Find files that matter to the current task
Retrieval	Document chunks, knowledge base content	Wrong recall, broken chunks	Balance recall and precision
Tool result	API output, shell output, search results	Too long, temporary or erroneous	Compress while preserving verifiable facts
Planner state	Current step, completion state, failure reason	Overlong plans and state drift	Keep the minimum state needed for the next step

Context is not one text blob. It is a composition of multiple sources. The hard part is cross-source selection, not optimizing one prompt paragraph.

7.5 Pruning, Summarization and Ordering

The context builder commonly performs three actions: pruning, summarization and ordering.

Pruning decides what does not enter context. It is the most direct token-reduction technique, but the risk is removing a critical fact. Pruning should not be based only on length. It should consider task goal, recent changes, file type, permissions and historical importance.

Summarization compresses long content into shorter content. It reduces tokens, but it can lose information. Technical docs, code diffs, tool outputs and error logs should not be summarized in the same way. A good summary preserves verifiable facts, identifiers, failure causes and constraints needed for the next step.

Ordering decides what the model sees first. LLMs are sensitive to position. Important information buried in the middle or at the end may be underused. The context builder should place task goals, constraints, latest state and strongest evidence where the model can use them.

7.6 Prompt Caching and Stable Prefixes

A lot of agent cost comes from repeatedly sending stable content: system instructions, tool definitions, output formats, project-level rules and terminology. These change little during an agent loop and are good candidates for stable prefixes.

Prompt caching reduces the cost of repeated prefixes. It does not change the agent's logic, but it does influence prompt structure. Stable content should be centralized, ordered consistently and rewritten only when necessary. Dynamic content should come later and contain only task-relevant information.

This resembles cache-friendly system design. Cache hits require stable keys, stable structure and predictable change boundaries. Agent prompts are similar. If each round injects changing timestamps, random phrasing or irrelevant logs into the prefix, prompt caching becomes less useful.

7.7 Failure Modes

Context engineering failures usually come from bad data-loading strategy, not from the model suddenly becoming worse.

First, under-recall. A key file, memory or tool result does not enter context, so the model guesses.

Second, over-recall. Too much irrelevant content enters context, distracting the model and increasing cost.

Third, lossy summarization. The summary removes constraints, edge cases or error details, and the model reasons from a damaged compression.

Fourth, bad ordering. The right information is present but placed where the model does not use it well.

Fifth, cache misses. Stable prefixes are rewritten unnecessarily, preventing prompt caching from helping.

7.8 Summary

This chapter positioned context engineering as query optimization. Agents should not chase unlimited context. They should select, filter, order and compress data the way query optimizers prepare execution.

This perspective explains why AGENTS.md matters. It is not just a note file. It is an index-like entry point that helps the agent find project knowledge. Chapter 8 discusses AGENTS.md as a prompt index.

Chapter 8. AGENTS.md as a Prompt Index

Chapter focus: explain why AGENTS.md is closer to a project-level index than to a normal README.

8.1 The Question

When an agent enters a project, it does not face one file. It faces a workspace: code, docs, configuration, tests, scripts, historical conventions and implicit workflows. Even a strong model becomes expensive, slow and inconsistent if it has to rediscover the whole project every time.

This is where AGENTS.md matters. It is not just a note for humans, and not merely a prompt for the model. From a systems perspective, it is a project-level prompt index: a small entry point that tells the agent where to begin, which rules matter and which files form the critical path.

This chapter explains why AGENTS.md behaves like an index, and what it should and should not contain.

8.2 README, Docs and AGENTS.md

A README usually serves human readers. It explains the project goal, installation and basic usage. Detailed documentation explains design, APIs and operational workflows. AGENTS.md serves the agent. Its goal is not to explain the whole project, but to help the agent locate relevant knowledge quickly.

This means AGENTS.md should not duplicate all documentation. It should point to where build commands live, how tests run, what style constraints are hard requirements, which directories contain core modules, which files should not be edited casually and where to start for common task types.

If the README is the project homepage, AGENTS.md is closer to a query entry point and routing table. It does not replace documentation. It helps the agent decide which documentation to load.

8.3 AGENTS.md as Prompt Index

The value of an index is using a small structure to locate large content. A database index does not store every field of every row, but it helps the query find relevant rows quickly. AGENTS.md is similar. It should not contain all project knowledge, but it should contain enough routing information to find the right places.

A project-level prompt index should answer several questions. What is the technology stack? What are the common commands? How do tests, formatting and release generation work? How are core directories organized? Where is task-specific documentation? Which conventions override local guesses? Which operations are risky?

If this information is scattered across the repository, the agent has to rediscover it every time. AGENTS.md centralizes the entry points, reducing context loading cost and lowering the chance of misreading project structure.

8.4 What a Good AGENTS.md Contains

Content type	Example	Purpose
Project role	API service, frontend app, data pipeline or documentation project	Establish task boundaries
Key directories	`src/`, `tests/`, `docs/`, `scripts/`	Locate files quickly
Common commands	Test, format, build, release generation	Avoid guessing commands
Code/doc conventions	Naming, formatting, terminology, chapter structure	Keep edits consistent
Risk boundaries	Do not edit generated files, rewrite chapters or delete releases	Avoid destructive changes
Task routing	Where to start for API fixes or documentation edits	Guide context selection

These entries should be concise, stable and actionable. AGENTS.md is not a long architecture document. It is a high-signal index.

8.5 What Does Not Belong in AGENTS.md

The main risk of AGENTS.md is bloat. If it becomes another manual, the agent still has to read a large amount of text and the index loses value.

Bad candidates for AGENTS.md include full business background, long tutorials, every API detail, all historical decisions, information already obvious from code, and temporary task lists that change frequently. Those belong in dedicated documents or issues, with AGENTS.md pointing to them.

In short, AGENTS.md should store "where to look" and "what must be obeyed," not all knowledge itself. Its purpose is to improve the context builder's hit rate, not to enlarge the prompt.

8.6 Layered AGENTS.md and Scope

Large projects may need multiple AGENTS.md files. The root file provides global rules. Subdirectory files provide local rules. Backend, frontend, mobile, documentation and data-pipeline areas may all have different commands and conventions.

This resembles configuration inheritance or routing tables. Global rules apply across the repository. Local rules apply only in a specific directory. When editing a file, the agent should

load the relevant AGENTS.md files along the path from the root to the target directory, not just one global file.

Scope matters. If every local rule is placed in the root AGENTS.md, the file bloats. If only local rules exist without global rules, the agent may miss project-wide constraints. Layered indexes balance the two.

8.7 Failure Modes

First, staleness. Commands, directories or conventions change but AGENTS.md does not, so the agent follows the wrong index.

Second, length. AGENTS.md becomes an encyclopedia. Loading cost rises and critical rules get buried.

Third, vagueness. It says "keep code quality high" or "remember tests" but gives no concrete commands or boundaries.

Fourth, conflict with source. AGENTS.md says one convention, while the code follows another, and the agent cannot tell which source to trust.

Fifth, no task routing. The file lists rules but does not tell the agent where to start for different types of work.

8.8 Summary

This chapter positioned AGENTS.md as a prompt index. It does not replace the README or full documentation. It gives the agent a low-cost, high-signal project entry point.

A good AGENTS.md helps the agent perform context routing: finding the rules, docs and code that matter for the current task. Chapter 9 continues with retrieval and context routing, showing why retrieval is not just similar-text search but choosing the right context path.

Chapter 9. Retrieval and Context Routing

Chapter focus: expand retrieval from similar-text search into context routing for agents.

9.1 The Question

RAG is often described as "retrieve, then generate." That definition is not wrong, but it is too narrow for agent systems. Real agents do not only face knowledge-base documents. They face project files, code, memory, tool results, task state, issues, logs and external systems.

Retrieval inside an agent should therefore not mean only vector similarity search. It is closer to context routing: based on task type, permissions, state and cost, the agent decides which information sources to read, what to read from them and how to place the result into model context.

This chapter asks: what role does retrieval actually play inside an agent? Why is "find similar text" only one part of the problem?

9.2 RAG Is One Read Path

Chapter 5 positioned RAG as one read path from storage into context. It is useful for large bodies of unstructured text such as docs, knowledge bases, manuals and historical records. The retrieval system finds relevant chunks, and the context builder places them into the prompt.

But an agent has many read paths beyond RAG. Querying a database, reading files, calling APIs, loading memory, reading task state and inspecting tool logs are also read paths. They may not use vector search, but they answer the same question: what external information does this model call need?

A more precise architecture statement is: RAG is one implementation technique inside context routing. It is not the entire context system.

9.3 Inputs to Context Routing

Context routing uses multiple signals to choose read paths.

Signal	Example	Effect
Task type	Coding, fact lookup, email summary, document edit, bug debugging	Prioritize code, docs, memory or tool results
Data shape	Structured table, long document, code, log, image	Choose SQL, full-text search, vector search or file read
Freshness	Current file, historical memory, real-time API	Decide whether cached data is safe or live reads are required
Permission	User auth, repository access, tool capability	Decide which sources can be accessed

Cost	Tokens, latency, API cost	Decide how much to read and whether to compress
Risk	Production impact, private data	Decide whether confirmation or audit is required

These signals show that retrieval is not just similarity ranking. Similarity is one score. Context routing is a system-level decision.

9.4 Value and Limits of Vector Retrieval

Vector retrieval is good at finding semantically similar content in large text collections. It is useful for documentation, historical discussions, similar issues and conceptual explanations. For agents, it can reduce the cost of understanding a project from scratch.

But vector retrieval has limits. First, it may return content that looks relevant but is stale or wrong. Second, it does not naturally understand permissions or task state. Third, it is not always better than SQL for structured queries. Fourth, it returns chunks, which may lose full context and causality.

Vector retrieval should therefore be combined with metadata filters, timestamps, permission checks, file paths, code structure and task state. Pure top-k similar chunks can easily send the wrong context to the model.

9.5 Hybrid Retrieval

Production agents often need hybrid retrieval. Different sources and methods complement each other. Keyword search is good for exact identifiers. Vector search is good for semantic similarity. Structured queries are good for state and permissions. File paths are good for project structure. Tool calls are good for live information.

For example, debugging a bug may require several reads. The agent may first search the error message as a keyword, then use file paths to locate the module, then read tests, then inspect recent changes, then load project conventions from memory. No single retrieval method solves the whole task.

The key is routing order. What to read first, what to read next, when to stop and when to escalate to a more expensive retrieval or model call all affect cost and quality.

9.6 Retrieval Results Are Not Final Context

Retrieval results are not the same as final context. They still need filtering, deduplication, ordering, compression and verification.

A common mistake is placing top-k chunks directly into the model. Those chunks may contain duplicates, stale content, conflicts or irrelevant details. The context builder should first decide which chunks actually support the task, then choose whether to summarize, quote, keep raw text or discard them.

This is especially true for code and tool outputs. Code snippets need function names, file paths and call relationships. Logs need timestamps, error codes and key stack frames. Tool outputs need verifiable facts and side-effect state. Different content needs different compression strategies.

9.7 Failure Modes

First, wrong source routing. The agent uses document retrieval when it should query a database, or loads old memory when it should read the current file.

Second, recall bias. Retrieved content is semantically similar but does not match the current task constraints.

Third, permission leakage. The agent loads information the current user should not see.

Fourth, freshness errors. The system uses a cache or stale index instead of reading current state.

Fifth, unclear stopping conditions. The agent keeps retrieving and appending context, increasing cost without improving quality.

These failures show that retrieval must be integrated with permissions, state, caching, audit and planning. It should not exist as a standalone module.

9.8 Summary

This chapter placed RAG and retrieval inside the broader frame of context routing. RAG is an important read path, but an agent's context system also includes databases, files, memory, tool results, task state and logs.

Useful retrieval does not merely find similar text. It chooses the right information source based on task, permission, state and cost, then turns the result into context the model can actually use. The next chapter returns to the cost model and discusses token reduction, distillation and tiered compute.

Plan for the Remaining Chapters

Part I now completes the first nine chapters of the foundational system perspective. Later chapters will continue the same line of reasoning into the cost boundary between token reduction and distillation, production reliability, multi-agent / multi-tenant scheduling, agent security and agent OS.

The next stage should also add three engineering themes that strengthen the book's differentiation. First, agent security should not remain a footnote. Operating-system security models will migrate into agent systems: prompt injection behaves more like an exploit, tool use needs capability boundaries, and code or external-system access needs sandboxing. Second, concurrent scheduling is where the OS analogy becomes most literal. Multi-agent, multi-tenant execution, task queues and resource isolation go beyond the single-task Planner-to-Scheduler analogy. Third, agents should be treated as production systems: they need SLAs, idempotency, optimistic locking, state machines, retries, degradation, escalation, observability, audit, replay and reconciliation. This production reliability perspective is the main difference from OS-analogy papers that focus mostly on architecture and runnable prototypes.

1. 1. Chapters 1-9 have established the core architecture line from LLMs and agents to memory / tools / planner, compute/storage separation, stateless agents, context engineering, prompt indexes and context routing.
2. 1. Chapter 10 will discuss token reduction, distillation, small models and tiered compute, with emphasis on separating token-count reduction from per-token compute-cost reduction.
3. 1. Chapter 11 will treat agents as production systems, focusing on idempotency, optimistic locking, state machines, retries, degradation, escalation, observability, audit, replay and reconciliation.
4. 1. Chapter 12 will cover multi-agent, multi-tenant and concurrent scheduling, moving the Planner-to-Scheduler analogy from single-task execution to resource contention and orchestration.
5. 1. Chapter 13 will add the missing agent security model, including prompt injection as exploit, tool capability boundaries, sandboxing, least privilege and audit.
6. 1. Chapter 14 will return to the agent operating system idea and connect memory, tools, planning, security, scheduling and production reliability into one system view.